RenderBridge AI 協作報告

1. 專案目標

本專案是針對軟體工程師面試需求完成的 SketchUp 渲染器插件原型。我的目標不是在三天內做出商業級算圖引擎，而是展示以下能力：

- 能將 SketchUp 插件拆成輕量前端與後端中繼服務。
- 能設計清楚的 API 串接流程。
- 能處理 SketchUp Ruby 單執行緒環境下的非同步等待問題。
- 能使用 VS Code、Git、GitHub 與 AI coding 工具完成可追蹤的開發流程。

專案名稱為 RenderBridge AI，架構為 SketchUp Ruby Plugin + FastAPI Middleware。FastAPI 端目前使用 Mock endpoint 模擬 AI 算圖延遲，未來可替換成 Stable Diffusion API、OpenAI API 或其他影像生成服務。

2. 我如何使用 AI 協助

我把 AI 定位成 pair-programmer，而不是完全代替我做決策的工具。我的做法是先描述面試題目、交付時間與技術限制，再要求 AI 按照小步驟協助：

1. 先建立乾淨的專案資料夾與 Git 初始化。

2. 再實作 FastAPI mock backend。

3. 接著建立 SketchUp Extension 標準骨架。

4. 再補 UI 與 JavaScript/Ruby callback。

5. 最後處理最重要的非同步 HTTP 與 SketchUp UI 不阻塞問題。

每完成一個 step，我都要求 commit 並 push 到 GitHub，讓開發歷程可以被審查，而不是只交出最後結果。

3. 我的判斷與 AI 建議不同之處

差異一：不是把所有功能都寫在 SketchUp Ruby 裡

一開始如果只追求最快完成，很容易把圖片擷取、prompt、HTTP request、算圖等待全部塞在 SketchUp Ruby plugin 裡。這樣雖然檔案少，但會讓 SketchUp 主程式承擔太多責任，也不容易替換真實 AI 算圖服務。

我的判斷是採用 Client-Server Architecture：

- SketchUp Plugin 只負責 UI、viewport capture、送 request、顯示結果。
- FastAPI Middleware 負責接收資料、模擬算圖流程、未來串接真實 AI provider。

這個決策讓專案更接近真實產品架構，也能展示後端 API 設計能力。

差異二：不接受同步 HTTP request 卡住 SketchUp

AI 在產生程式時很容易直接使用 Net::HTTP.post 或同步 request，這在一般 CLI 程式可以接受，但在 SketchUp Ruby plugin 裡會卡住 UI。面試題也特別要求不能讓 SketchUp 主執行緒被 5-10 秒的算圖等待阻塞。

我的修正方向是：

- `view.write_image` 保留在 SketchUp 主執行緒，因為 SketchUp API 通常不應該在背景 thread 操作。
- HTTP I/O 放入 Ruby `Thread.new`。
- 用共享 job 狀態紀錄 pending/done/error。
- 主執行緒透過 `UI.start_timer` 定期檢查完成的 job，再更新 HtmlDialog。

這個做法避免 SketchUp 在等待後端時失去操作能力，也讓程式責任分離更清楚。

差異三：保留 Mock API，而不是假裝已完成真實 AI 算圖

在三天面試時間內，若強行串接外部 AI 算圖服務，會花很多時間處理金鑰、額度、模型參數、網路錯誤與結果穩定性。這可能稀釋面試官真正想看的重點。

我的判斷是先完成 Mock Endpoint：

- API request/response shape 先固定。
- 用 `asyncio.sleep(5)` 模擬真實雲端算圖延遲。
- 回傳 Base64 圖片給前端顯示。

這樣面試時可以清楚說明：目前 mock 的位置就是未來替換真實 AI provider 的擴充點。

4. 我如何引導 AI 解決問題

我沒有一次要求 AI 產生整個專案，而是把任務拆成可驗證的小步：

- Step 0 只做 repo 與資料夾結構，避免一開始混入功能變更。
- Step 1 只做 backend，先確認 API 可以獨立啟動與測試。
- Step 2 只做 SketchUp extension shell，先確認選單與 HtmlDialog 的責任。
- Step 3 只做 UI 與 callback，先確認 JS 到 Ruby 的通路。
- Step 4 才處理 viewport capture、HTTP request、timer polling。

這樣做的好處是每一步都能 commit，問題也能被限制在小範圍內。當 AI 給出的方向可能造成阻塞或耦合太重時，我會要求它改成非同步設計，並把 SketchUp API 與 HTTP I/O 分開。

5. 技術亮點

- 使用 SketchUp 官方 Extension 註冊方式建立插件入口。
- 使用 `UI::HtmlDialog` 建立互動介面。
- 使用 JavaScript `window.sketchup.*` callback 與 Ruby 溝通。

- 使用 `view.write_image` 擷取 SketchUp viewport。
- 使用 Base64 封裝圖片資料，方便 JSON API 傳遞。
- 使用 FastAPI 建立 `/health` 與 `/api/render`。
- 使用 `asyncio.sleep(5)` 模擬真實 AI 算圖延遲。
- 使用 Ruby thread + `UI.start_timer` 避免 SketchUp UI 被 HTTP request 阻塞。

6. 目前限制與後續改進

目前版本仍是 MVP，限制如下：

- FastAPI render endpoint 是 mock，尚未串接真實 AI image provider。
- SketchUp runtime 需要在本機 SketchUp 內完整驗證。
- 目前回傳圖是 mock 結果，尚未實作風格化濾鏡或真實算圖。
- 錯誤處理已涵蓋基本 backend error，但尚未做 retry、request cancellation 或 render history。

後續可以改進：

- 將 backend provider 抽象成 adapter，支援 OpenAI、Stable Diffusion 或其他服務。
- 加入 job id 查詢 API，讓 backend 更接近真實長任務架構。
- 加入 render history 與輸出檔案管理。
- 在 HtmlDialog 顯示原圖與結果圖 side-by-side。

7. 總結

這個專案最重要的成果不是 mock 圖片本身，而是我如何在短時間內做出合理架構，並避免 SketchUp plugin 開發中常見的同步阻塞問題。AI 幫助我加速產生程式碼與指令，但架構切分、非同步策略、mock 範圍控制與 Git commit 節奏，是我根據題目限制做出的工程判斷。